

Министерство науки и высшего образования Российской
Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО
ITMO University

ЛАБОРАТОРНАЯ РАБОТА №6

По дисциплине Администрирование платформ на ОС Linux

Тема работы Docker

Обучающиеся Пухарев Сергей Валерьевич

Факультет прикладной информатики

Группа К3220

Направление подготовки 11.03.02 Инфокоммуникационные технологии и
системы связи

Образовательная программа Программирование в
инфокоммуникационных системах

Обучающийся

(дата)

(подпись)

Пухарев С.В.
(Ф.И.О.)

Руководитель

(дата)

(подпись)

Береснев А.Д.
(Ф.И.О.)

СОДЕРЖАНИЕ

Стр.

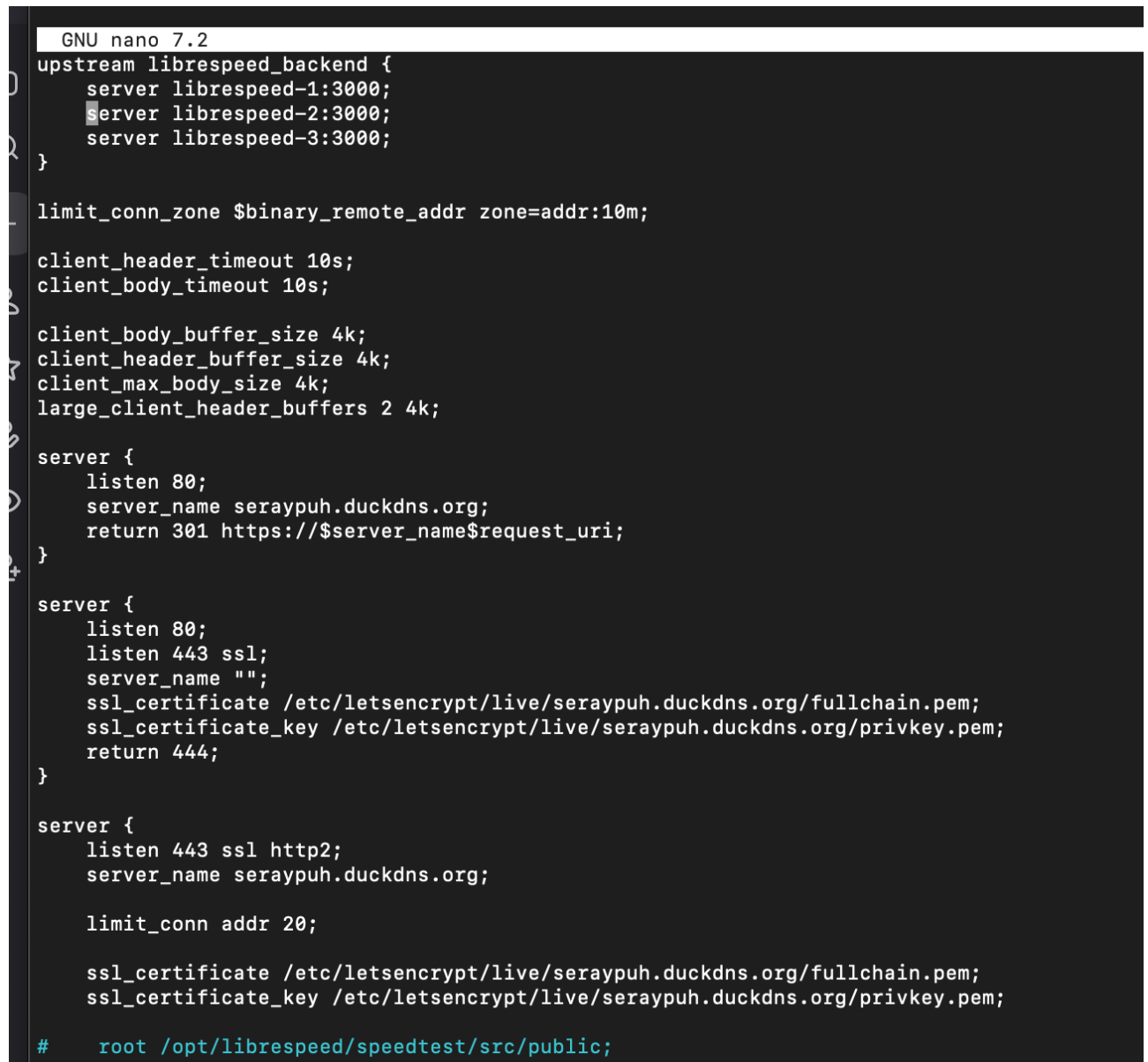
ВВЕДЕНИЕ	3
1 ПОДГОТОВКА.....	4
2 ЗАДАЧА.....	8
3 ВОПРОСЫ.....	14
ЗАКЛЮЧЕНИЕ	15

ВВЕДЕНИЕ

Цель работы: Научиться переводить инфраструктуру, развёрнутую на процессах Linux в декларативное окружение на базе Docker Compose с использованием собственных Dockerfile.

1 ПОДГОТОВКА

На рисунке 1.1 представлены изменения в файле `librespeed.conf` (в будущем – `nginx.conf`).



```
GNU nano 7.2
upstream librespeed_backend {
    server librespeed-1:3000;
    server librespeed-2:3000;
    server librespeed-3:3000;
}

limit_conn_zone $binary_remote_addr zone=addr:10m;

client_header_timeout 10s;
client_body_timeout 10s;

client_body_buffer_size 4k;
client_header_buffer_size 4k;
client_max_body_size 4k;
large_client_header_buffers 2 4k;

server {
    listen 80;
    server_name seraypuh.duckdns.org;
    return 301 https://$server_name$request_uri;
}

server {
    listen 80;
    listen 443 ssl;
    server_name "";
    ssl_certificate /etc/letsencrypt/live/seraypuh.duckdns.org/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/seraypuh.duckdns.org/privkey.pem;
    return 444;
}

server {
    listen 443 ssl http2;
    server_name seraypuh.duckdns.org;

    limit_conn addr 20;

    ssl_certificate /etc/letsencrypt/live/seraypuh.duckdns.org/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/seraypuh.duckdns.org/privkey.pem;

#    root /opt/librespeed/speedtest/src/public;
```

Рисунок 1.1 — Librespeed.conf

Далее был установлен Docker compose (рисунок 1.2).

```
root@pukharevsv:~# docker --version
[docker compose version
Docker version 29.4.0, build 9d7ad9f
Docker Compose version v5.1.2
root@pukharevsv:~# sudo usermod -aG docker $USER
newgrp docker
[# или выйдите из системы и зайдите снова
root@pukharevsv:~# docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:f9078146db2e05e794366b1bfe584a14ea6317f44027d10ef7dad65279026885
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

root@pukharevsv:~# █
```

Рисунок 1.2 — Docker compose

Была создана файловая система, в соответствии с заданием (рисунок 1.3).

```

├── CHANGELOG.md
├── LICENSE.md
├── README.md
├── README_js.md
├── bin
│   └── uuid
├── index.js
├── lib
│   ├── bytesToUuid.js
│   ├── md5-browser.js
│   ├── md5.js
│   ├── rng-browser.js
│   ├── rng.js
│   ├── sha1-browser.js
│   ├── sha1.js
│   └── v35.js
├── package.json
├── v1.js
├── v3.js
├── v4.js
├── v5.js
├── vary
│   ├── HISTORY.md
│   ├── LICENSE
│   ├── README.md
│   ├── index.js
│   └── package.json
├── verror
│   ├── CHANGES.md
│   ├── CONTRIBUTING.md
│   ├── LICENSE
│   ├── README.md
│   ├── lib
│   │   └── verror.js
│   └── package.json
├── xtend
│   ├── LICENSE
│   ├── README.md
│   ├── immutable.js
│   ├── mutable.js
│   ├── package.json
│   └── test.js
├── package-lock.json
├── package.json
├── src
│   ├── Helpers.js
│   ├── SpeedTest.js
│   └── backup_librespeed.sql
├── public
│   ├── example-multipleServers-pretty.html
│   ├── index.html
│   ├── speedtest.js
│   └── speedtest_worker.js
├── yarn.lock
├── nginx
│   ├── Dockerfile
│   └── nginx.conf
├── postgres
│   ├── Dockerfile
│   └── init.sql
└── 262 directories, 1091 files
root@pukharevsv:~#

```

Рисунок 1.3 — Файловая система

Были выключены БД, nginx, бэкенды и юниты из прошлых лабораторных работ (рисунок 1.4).

```

202 directories, 1671 files
root@pukharevsv:~# sudo systemctl stop nginx-custom.service librespeed@8888.service librespeed@8889.service librespeed@8890.service postgres-backup.timer postgres-backup.service mymsg.service
O
sudo systemctl disable nginx-custom.service librespeed@8888.service librespeed@8889.service librespeed@8890.service postgres-backup.timer mymsg.service
Removed "/etc/systemd/system/timers.target.wants/postgres-backup.timer".
Removed "/etc/systemd/system/multi-user.target.wants/nginx-custom.service".
Removed "/etc/systemd/system/multi-user.target.wants/librespeed@8889.service".
Removed "/etc/systemd/system/multi-user.target.wants/librespeed@8888.service".
Removed "/etc/systemd/system/multi-user.target.wants/librespeed@8890.service".
Removed "/etc/systemd/system/multi-user.target.wants/mymsg.service".
root@pukharevsv:~# sudo systemctl stop myapp.target
root@pukharevsv:~# sudo systemctl stop postgresql.service
root@pukharevsv:~# sudo systemctl stop nginx-custom.service
root@pukharevsv:~# sudo systemctl stop librespeed@8888.service librespeed@8889.service librespeed@8890.service
root@pukharevsv:~# sudo systemctl stop nginx.service
root@pukharevsv:~# sudo systemctl disable postgresql.service
sudo systemctl disable nginx-custom.service
sudo systemctl disable librespeed@8888.service librespeed@8889.service librespeed@8890.service
Synchronizing state of postgresql.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install disable postgresql
Removed "/etc/systemd/system/multi-user.target.wants/postgresql.service".
root@pukharevsv:~# sudo systemctl disable myapp.target
Removed "/etc/systemd/system/multi-user.target.wants/myapp.target".
root@pukharevsv:~# systemctl daemon-reload
root@pukharevsv:~# systemctl status postgresql.service
O postgresql.service - PostgreSQL RDBMS
   Loaded: loaded (/etc/systemd/system/postgresql.service; disabled; preset: enabled)
   Active: inactive (dead) since Wed 2020-04-15 09:06:06 UTC; 1min 26s ago
     Duration: 1h 12min 37.491s
    Main PID: 752 (code=exited, status=0/SUCCESS)
       CPU: 1ms

Apr 15 07:53:28 pukharevsv systemd[1]: Starting postgresql.service - PostgreSQL RDBMS...
Apr 15 07:53:28 pukharevsv systemd[1]: Finished postgresql.service - PostgreSQL RDBMS.
Apr 15 09:06:06 pukharevsv systemd[1]: postgresql.service: Deactivated successfully.
Apr 15 09:06:06 pukharevsv systemd[1]: Stopped postgresql.service - PostgreSQL RDBMS.
root@pukharevsv:~# █

```

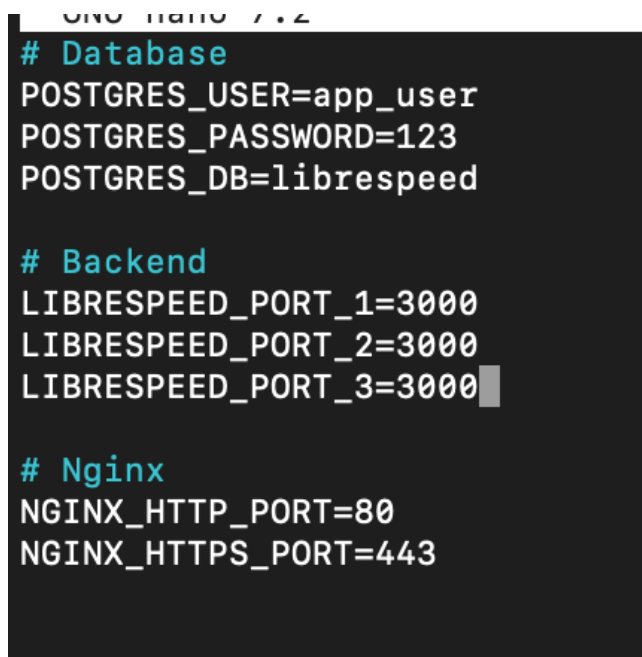
Рисунок 1.4 — Выключение процессов

2 ЗАДАЧА

Было необходимо переработать инфраструктуру на использование Docker Compose:

- Все компоненты должны быть описаны в виде собственных Dockerfile.
- Три сервиса librespeed должны именоваться librespeed-1, librespeed-2, librespeed-3. Они должны быть в одной сети.
- Все переменные окружения описаны в .env.
- Для nginx использован volumes - nginx_logs:/var/log/nginx/etc/letsencrypt:/etc/letsencrypt:ro. Скрипт бэкапа логов использует volume nginx_logs.
- Для postgres использован volume pg_data:/var/lib/postgresql/data. Использован depends_on и healthcheck.

На рисунке 2.1 представлен .env-файл.

A screenshot of a terminal window showing the contents of a .env file being edited with GNU nano 7.2. The file contains three sections: Database, Backend, and Nginx, each with specific environment variables.

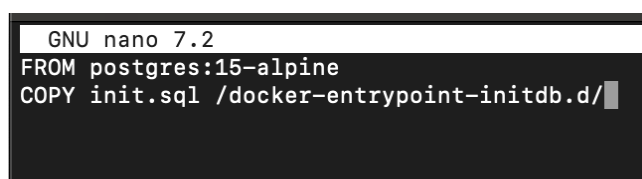
```
GNU nano 7.2
# Database
POSTGRES_USER=app_user
POSTGRES_PASSWORD=123
POSTGRES_DB=librespeed

# Backend
LIBRESPEED_PORT_1=3000
LIBRESPEED_PORT_2=3000
LIBRESPEED_PORT_3=3000

# Nginx
NGINX_HTTP_PORT=80
NGINX_HTTPS_PORT=443
```

Рисунок 2.1 — .env

На рисунке 2.2 представлен dockerfile postres-a.

A screenshot of a terminal window showing the contents of a Dockerfile being edited with GNU nano 7.2. The file contains two instructions: FROM and COPY.

```
GNU nano 7.2
FROM postgres:15-alpine
COPY init.sql /docker-entrypoint-initdb.d/
```

Рисунок 2.2 — Dockerfile postres

На рисунке 2.3 представлен dockerfile librespeed-a.

```
GNU nano 7.2
FROM node:18-alpine

WORKDIR /usr/src/app

COPY app/ .

CMD ["sh", "-c", "node src/SpeedTest.js ${PORT}"]
```

Рисунок 2.3 — Dockerfile librespeed

На рисунке 2.4 представлен dockerfile nginx-a.

```
GNU nano 7.2
FROM nginx:stable-alpine

COPY nginx.conf /etc/nginx/conf.d/default.conf
```

Рисунок 2.4 — Dockerfile nginx

На рисунке 2.5 представлен dockerfile backup-logs.

```
FROM amazon/aws-cli:latest

RUN yum install -y bash && yum clean all

COPY backup-nginx-logs.sh /usr/local/bin/
RUN chmod +x /usr/local/bin/backup-nginx-logs.sh

ENTRYPOINT []

CMD ["/usr/local/bin/backup-nginx-logs.sh"]
```

Рисунок 2.5 — Dockerfile backup-logs

На рисунке 2.6 представлен dockerfile backup-db.

```
GNU nano 7.2
FROM postgres:15-alpine

RUN apk add --no-cache bash

COPY backup.sh /usr/local/bin/
RUN chmod +x /usr/local/bin/backup.sh

CMD ["/usr/local/bin/backup.sh"]
```

Рисунок 2.6 — Dockerfile backup-db

На рисунках 2.7 и 2.8 представлен yml-файл docker compose.

```

version: '3.8'

services:
  postgres_db:
    build: ./postgres
    container_name: myapp-postgres
    env_file:
      - .env
    volumes:
      - pg_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 30s
    restart: unless-stopped
    networks:
      - app-network

  librspeed-1:
    environment:
      - PORT=8888
      - PG_HOST=postgres_db
      - PG_USER=${POSTGRES_USER}
      - PG_PASSWORD=${POSTGRES_PASSWORD}
      - PG_DATABASE=${POSTGRES_DB}
    build: ./librspeed
    container_name: librspeed-1
    networks:
      - app-network
    restart: unless-stopped

  librspeed-2:
    environment:
      - PORT=8889
      - PG_HOST=postgres_db
      - PG_USER=${POSTGRES_USER}
      - PG_PASSWORD=${POSTGRES_PASSWORD}
      - PG_DATABASE=${POSTGRES_DB}
    build: ./librspeed
    container_name: librspeed-2
    networks:
      - app-network
    restart: unless-stopped

  librspeed-3:
    environment:
      - PORT=8890
      - PG_HOST=postgres_db
      - PG_USER=${POSTGRES_USER}
      - PG_PASSWORD=${POSTGRES_PASSWORD}
      - PG_DATABASE=${POSTGRES_DB}
    build: ./librspeed
    container_name: librspeed-3
    networks:
      - app-network
    restart: unless-stopped

```

Рисунок 2.7 — Yml

```

nginx:
  build: ./nginx
  container_name: myapp-nginx
  ports:
    - "${NGINX_HTTP_PORT}:80"
    - "${NGINX_HTTPS_PORT}:443"
  volumes:
    - nginx_logs:/var/log/nginx
    - /etc/letsencrypt:/etc/letsencrypt:ro
  depends_on:
    librespeed-1:
      condition: service_started
    librespeed-2:
      condition: service_started
    librespeed-3:
      condition: service_started
    postgres_db:
      condition: service_healthy
  networks:
    - app-network
  restart: unless-stopped

backup-logs:
  build: ./backup-logs
  container_name: myapp-backup-logs
  volumes:
    - nginx_logs:/var/log/nginx:ro
  env_file:
    - .env
  depends_on:
    - nginx
  networks:
    - app-network
  restart: "no"

backup-db:
  build: ./backup-db
  container_name: myapp-backup-db
  volumes:
    - ./backups:/backup
  env_file:
    - .env
  depends_on:
    postgres_db:
      condition: service_healthy
  networks:
    - app-network
  restart: "no"

volumes:
  pg_data:
  nginx_logs:

networks:
  app-network:
    driver: bridge

```

Рисунок 2.8 — Yml

На рисунке 2.9 представлен юнит для автозапуска yml-файла docker compose при старте системы.

```

[Unit]
Description=Docker Compose project
Requires=docker.service
After=docker.service

[Service]
Type=oneshot
RemainAfterExit=yes
WorkingDirectory=/root/project
ExecStart=/usr/bin/docker compose up -d
ExecStop=/usr/bin/docker compose down
ExecReload=/usr/bin/docker compose restart

[Install]
WantedBy=multi-user.target

```

Рисунок 2.9 — Юнит

На рисунке 2.10 представлен запуск и проверка юнита.

```

root@pukharevsv:~# sudo nano /etc/systemd/system/project-containers.service
root@pukharevsv:~# sudo systemctl daemon-reload
root@pukharevsv:~# sudo systemctl enable project-containers.service
Created symlink /etc/systemd/system/multi-user.target.wants/project-containers.service → /etc/systemd/system/project-containers.service.
root@pukharevsv:~# sudo systemctl start project-containers.service
root@pukharevsv:~# sudo systemctl status project-containers.service
● project-containers.service - Docker Compose project
   Loaded: loaded (/etc/systemd/system/project-containers.service; enabled; preset: enabled)
   Active: active (exited) since Wed 2026-04-15 10:55:03 UTC; 5s ago
     Process: 17879 ExecStart=/usr/bin/docker compose up -d (code=exited, status=0/SUCCESS)
    Main PID: 17879 (code=exited, status=0/SUCCESS)
       CPU: 104ms

Apr 15 10:55:01 pukharevsv docker[17892]: Container myapp-nginx Running
Apr 15 10:55:01 pukharevsv docker[17892]: Container myapp-postgres Waiting
Apr 15 10:55:01 pukharevsv docker[17892]: Container myapp-postgres Waiting
Apr 15 10:55:02 pukharevsv docker[17892]: Container myapp-postgres Healthy
Apr 15 10:55:02 pukharevsv docker[17892]: Container myapp-backup-logs Starting
Apr 15 10:55:02 pukharevsv docker[17892]: Container myapp-postgres Healthy
Apr 15 10:55:02 pukharevsv docker[17892]: Container myapp-backup-db Starting
Apr 15 10:55:03 pukharevsv docker[17892]: Container myapp-backup-logs Started
Apr 15 10:55:03 pukharevsv docker[17892]: Container myapp-backup-db Started
Apr 15 10:55:03 pukharevsv systemd[1]: Finished project-containers.service - Docker Compose project.
root@pukharevsv:~#

```

Рисунок 2.10 — Запуск

На рисунке 2.11 представлено убийство одного из процессов librespeed.

```

myapp-postgres project-postgres_db docker-entrypoint.sh postgres_db 49 seconds ago Up 38 seconds (healthy) 5432/tcp
root@pukharevsv:~/project# docker compose stop librespeed-2
WARN[0000] /root/project/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] stop 1/1
✔ Container librespeed-2 Stopped
root@pukharevsv:~/project#

```

Рисунок 2.11 — Убийство процесса

На рисунке 2.12 представлена работоспособность приложения.

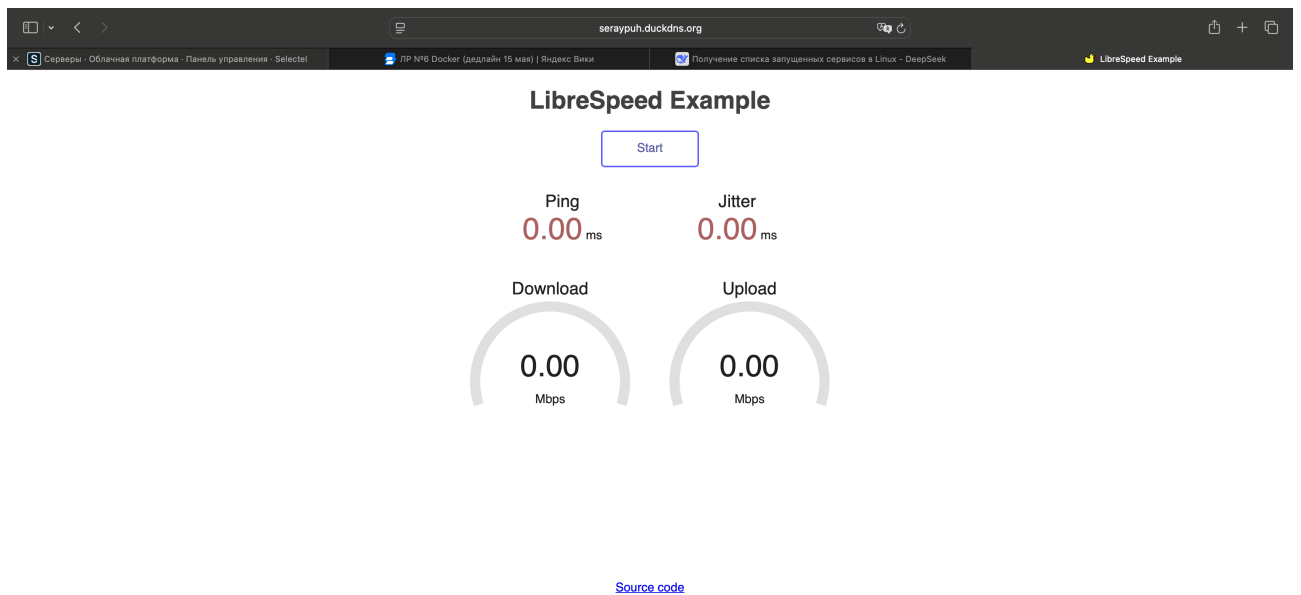


Рисунок 2.12 — Всё работает

3 ВОПРОСЫ

1. Потому что в Docker-контейнере статические файлы не хранятся локально на хосте, а отдаются самими бэкендами.
2. Docker Compose создаёт изолированную сеть, в которой контейнеры доступны друг другу по имени сервиса yaml-файла, а не по IP. Встроенный DNS-резолвер автоматически преобразует имя сервиса в его IP-адрес.

ЗАКЛЮЧЕНИЕ

В ходе выполнения данной лабораторной работы были получены навыки перевода инфраструктуры, развёрнутой на процессах Linux в декларативное окружение на базе Docker Compose с использованием собственных Dockerfile.